

Uma Visão Geral do Pensamento Computacional

Sibel Kılıçarslan Cansu, PhD 1

Fatih Küryat Cansu²

1 Faculdade de Ciências Naturais da Universidade Abant İzzet Baysal

² Instituto Universitário de Ciências da Educação Bahçeşehir

DOI: [10.21585/ijcses.v3i1.53](https://doi.org/10.21585/ijcses.v3i1.53)

Abstrato

Computadores e dispositivos inteligentes tornaram-se itens básicos onipresentes em nossas vidas. Computadores e dispositivos controlados por computador são usados em todas as indústrias, desde medicina até engenharia e produção têxtil. Um campo onde os computadores se espalharam inevitavelmente é a educação, e um pré-requisito para controlar os computadores, ou aumentar o nível e a eficiência do nosso controle sobre eles, é tornar a interação humano-computador tão eficiente quanto possível. Este processo de utilização eficiente e eficaz do computador, conhecido como "Pensamento semelhante ao de um computador" ou "Pensamento computacional", é visto como um campo com potencial para apoiar o desenvolvimento individual e social no nosso mundo em rápido progresso e para proporcionar benefícios econômicos significativos. Os conceitos fundamentais e o escopo deste campo foram delineados de diversas maneiras por diferentes pesquisadores. Da mesma forma, os pesquisadores também apresentaram pontos de vista críticos distintos e benefícios potenciais do pensamento computacional. Este estudo visa primeiro definir o conceito de pensamento computacional referenciando a literatura de origem, depois analisar os objetivos de certas críticas ao campo e discutir os elementos fundamentais do pensamento computacional e da pesquisa contemporânea sobre esses elementos.

Palavras-chave: pensamento computacional, pensamento computacional, pensamento computacional-informático

1. Introdução

"Computador" como palavra refere-se a um dispositivo que "computa", localizado em turco como "bilgisayar" pelo Prof. Dr. Aydın Köksal (Keser, 2011: p.88). No entanto, é difícil afirmar o mesmo sobre o "pensamento computacional", que é localizado de diversas maneiras pelos investigadores. Özden et al. (2015) usam "*bilgisayarca düşünme*", enquanto Yesan, Özçınar e Tanyeri (2017) preferem "*hesaplamalı düşünme*". Çınar e

Enquanto isso, Tüzün (2017) usou "*bilgi sayımsal düşünme*" e "*bilgi işlemsel düşünme*" em seu artigo.

Este estudo usará principalmente "*bilgi işlemsel düşünme*" (Pensamento Computacional). A presença de tentativas de localização tão diversas é natural. Como Piaget (Binguier, 1980: p.57) especificou, a definição dos termos ocorre após a criação dos termos na investigação científica.

A novidade deste campo, levando à falta de uniformidade no jargão e à divergência cotidiana de termos de uso comum, pode ser a explicação deste fenômeno. Uma diferenciação semelhante é observada na divisão ciência da computação/informática que separa os pesquisadores da área. Considerando que as fontes europeias preferem o termo "informática", colocando a informação antes dos dispositivos utilizados para a processar; Os investigadores americanos parecem preferir "ciência da computação" como termo para este campo (Kalelioğlu, Gülbahar e Kukul, 2016). Porém, apesar das diferenças na terminologia, observa-se que o foco fundamental deste campo são os princípios básicos da ciência da computação e sua interação com a humanidade.

2. A História do Pensamento Computacional

Embora o pensamento computacional seja amplamente considerado como tendo começado pelo artigo de Wing (2006) sobre o assunto, ele foi referenciado pela primeira vez por Papert (1996), como "pensamento processual". Papert, então no Departamento de Matemática do MIT, no decorrer de sua pesquisa sobre o uso de computadores e software na resolução de problemas geométricos, afirmou que o pensamento computacional poderia ser empregado na definição da relação entre um problema e sua solução e na estruturação de dados. Papert e seus colegas desenvolveram a linguagem de programação LOGO na década de 1960. O principal objetivo desta linguagem era ajudar os alunos a pensar matemática e logicamente. LOGO era, em sua essência, uma linguagem construtivista, aceitando a aprendizagem como uma atividade fundamentalmente individual e explicando-a em termos piagetianos. A individualização deste conceito por Papert (1991: p.1) resultou na noção de aprender fazendo. Papert's

a adoção desta filosofia não é surpreendente, considerando a sua experiência de trabalho ao lado de Piaget no Centro de Epistemologia Genética em Genebra entre 1958 e 1963. LOGO foi assim concebido como um ambiente propício e favorável à aprendizagem piagetiana (Logo, 2015).



Figura 1. Seymour Papert e o robô Turtle baseado em LOGO.

O LOGO e o ethos construtivista por trás dele foram considerados como tendo o potencial de transformar a educação quando a linguagem foi introduzida pela primeira vez. No entanto, este potencial não se concretizou, à medida que o construtivismo perdeu gradualmente força nos sistemas educativos do Reino Unido e dos EUA (Agalianos, Noss, e Whitty, 2001: p.497). Esta perda não foi inédita, pois outras linguagens de programação como PLATO (Lógica Programada para Operações Automáticas), CAI (Instrução Assistida por Computador), CBT (Treinamento Baseado em Computador) e CAL (Aprendizagem Assistida por Computador) também enfrentaram o mesmo destino (Etherington, 2017).).

3. Definindo Pensamento Computacional

Como o pensamento computacional é um campo recém-nascido, sua definição varia de pesquisador para pesquisador. Devido a esta variação entre acadêmicos, este artigo considerará definições práticas oferecidas por organizações como ISTE (Sociedade Internacional de Tecnologia na Educação) e CSTA (Associação de Professores de Ciência da Computação), além daquelas determinadas pelos próprios acadêmicos. Wing (2006, p.33) define pensamento computacional como

“O pensamento computacional envolve resolver problemas, projetar sistemas e compreender o comportamento humano, recorrendo aos conceitos fundamentais da ciência da computação.”. No entanto, após novas revisões [já que o artigo original tinha 4 páginas e muitos tópicos não foram totalmente explorados.], uma definição diferente foi aceita em 2011.

Segundo Wing (2011), o pensamento computacional é definido como “O pensamento computacional são os processos de pensamento envolvidos na formulação de problemas e suas soluções para que as soluções sejam representadas de uma forma que possa ser efetivamente executada por um agente de processamento de informações”. A Tabela 1 apresenta as diversas definições de pensamento computacional empregadas pela academia contemporânea.

Tabela 1. Definições contrastantes de pensamento computacional.

Definição	Fonte
...os processos de pensamento envolvidos na formulação de problemas e suas soluções, para que as soluções sejam representadas de uma forma que possa ser efetivamente executada por um agente de processamento de informações.	(Cuny, Snyder, Wing, 2010 akt. Wing, 2011, p.20)
O pensamento computacional são os processos de pensamento usados para formular um problema e expressar sua solução ou soluções em termos que um computador possa aplicar de forma eficaz.	Asa (2014)
O processo mental de abstração de problemas e criação de soluções automatizáveis.	Yadav et al. (2014)
O pensamento computacional é o processo de reconhecer aspectos da computação no mundo que nos rodeia e de aplicar ferramentas e técnicas da Ciência da Computação para compreender e raciocinar sobre sistemas e processos naturais e artificiais.	Furber (2012)

O pensamento computacional tem uma longa história na ciência da computação. Conhecido nas décadas de 1950 e 1960 como “pensamento algorítmico”, significa uma orientação mental para formular problemas como conversões de alguma entrada em saída e procurar algoritmos para realizar as conversões. Hoje, o termo foi expandido para incluir o pensamento com muitos níveis de abstração, o uso da matemática para desenvolver algoritmos e o exame de quão bem uma solução se adapta a diferentes tamanhos de problemas. Denning (2009)

...[Pensamento Computacional] é ensiná-los a pensar como um economista, um físico, um artista, e a compreender como usar a computação para resolver seus problemas, para criar e descobrir novas questões que podem ser exploradas de forma frutífera. Hemendinger (2010)

Essas definições tendem a se concentrar nos desempenhos e processos cognitivos dos indivíduos. Assim, podemos concluir que as atividades baseadas no pensamento computacional visam essencialmente melhorar as competências cognitivas e apoiar os processos de ensino e aprendizagem dos indivíduos afetados. Pesquisadores da área também realizaram workshops com o objetivo de estabelecer a verdadeira natureza e uma definição funcional para o pensamento computacional. Alguns destes workshops concluíram que uma abordagem rigorosa e consistente definição beneficiaria o campo (ByD Workshop Committee, 2011). Por outro lado, alguns investigadores sustentaram que a tentativa de definir o pensamento computacional em termos claros é desnecessária e que deveria ser aplicado esforço no estabelecimento das relações internas dentro do corpus do pensamento computacional (Voogt et al., 2015: p.726):

“Não existe uma definição clara para a TC e a principal tensão na tentativa de definir a TC tem a ver com a definição das competências essenciais da TC versus as competências mais periféricas. Argumentamos que, para efeitos de conceptualização e integração da PC na educação, não devemos tentar dar uma definição definitiva de PC, mas sim tentar encontrar semelhanças e relações nas discussões sobre PC (Voget et al., 2015: p. 726).

Embora um conceito geral de pensamento computacional possa ser estabelecido com base nestas definições, elas oferecem poucos insights sobre como o pensamento computacional deve ser aplicado na prática no campo da educação. Definições práticas de pensamento computacional e seus constituintes são necessárias antes que metas de desempenho e programas educacionais possam ser criados em sala de aula. CSTA e ISTE forneceram rubricas de atividades para pensamento computacional nos anos de 2011, 2015 e 2016. A Tabela 2 é uma lista dessas atividades, classificadas de acordo com palavras-chave.

Tabela 2. Atividades práticas de pensamento computacional, com curadoria do ISTE.

Palavras-chave	Fonte
Formulação, organização, análise, modelagem, abstrações, pensamento algorítmico, automação, eficiência, generalização, transferência	IST (2011)
Criatividade, pensamento algorítmico, pensamento crítico, resolução de problemas, cooperação	ISTE (2015; Oden et al. 2015)
Análise de dados, pensamento abstrato, pensamento algorítmico, modelagem, representação de dados, divisão de problemas em componentes, automação	ISTE (2016) (Definição de Pensador Computacional)

Como mostram estas definições, as listas de actividades fornecem um quadro para os educadores, delineando os resultados educativos que devem almejar e delineando métodos de avaliação e avaliação desses resultados. Por exemplo, um educador que utilize estas rubricas saberia que ensinar ferramentas de programação visual, como Scratch ou KODU, nas aulas, não se destina apenas a ajudar os alunos a divertirem-se enquanto criam jogos de computador; Eles também usariam a experiência como meio para incutir alguns dos conceitos e habilidades descritos na Tabela 2.

4. Componentes do Pensamento Computacional

Os componentes fundamentais do pensamento computacional também são fonte de divergência entre pesquisadores. A fim de estabelecer uma linha de base para análises posteriores, os componentes utilizados por vários investigadores foram fornecidos na Tabela 3.

Tabela 3. Componentes do Pensamento Computacional

Componentes	Fonte
Abstração, Algoritmos, Automação, Problema Decomposição, Paralelização, Simulação	Barr e Stephenson (2011)
Abstração, Automação, Análise	Lee et al. (2011)
Abstração, Pensamento Algorítmico, Decomposição, Avaliação, Generalização	Selby e Woollard (2013)
Abstração, Algoritmos, Decomposição, Depuração, Generalização	Angeli et al. (2016)
Abstração, Algoritmos, Automação, Problema Decomposição, Generalização	Asa (2006, 2008, 2011)

Embora os componentes exatos possam ser diferentes, acreditamos que os conceitos essenciais que eles representam são amplamente uniformes em todo o campo. As habilidades de pensamento computacional são essencialmente o conjunto de habilidades necessárias para converter problemas complexos, confusos e parcialmente definidos do mundo real em uma forma que um computador estúpido possa resolver sem a ajuda adicional de um ser humano (BCS, 2014, p.3). Como tal, este artigo utilizará as definições de abstração, decomposição de problemas, pensamento algorítmico, automação e generalização dentre os componentes fornecidos. Essas definições podem ser listadas como (Humphreys, 2015):

- A *abstração* facilita a reflexão sobre problemas ou sistemas. Abstração é o processo de fazer uma artefato mais compreensível através da redução de detalhes desnecessários e número de variáveis; levando, portanto, a soluções mais simples. Um dos exemplos mais conhecidos disso é o exemplo do metrô de Londres, fornecido por Humphreys (2015). O mapa do metrô de Londres fornece informações suficientes para o viajante navegar na rede subterrânea sem a carga desnecessária de informações como distância e posição geográfica exata. É uma representação que contém precisamente as informações necessárias para planejar uma rota de uma estação a outra – e nada mais. Exemplos semelhantes podem ser fornecidos para outras disciplinas, permitindo uma melhor compreensão do conceito (Wing, 2008):
 - Problemas verbais e baseados em histórias em matemática, como taxas de preenchimento de piscinas, áreas a serem cercadas e cálculos contábeis, são essencialmente um exercício de abstração para os alunos, onde eles são obrigados a separar dados relevantes e irrelevantes e declarar suas soluções de forma simbólica. linguagem de álgebra, geometria ou aritmética.
 - Em geografia, os alunos fazem uso de mapas especializados (físicos, topográficos, políticos, turísticos etc.), ignorando muitos aspectos da geografia do mundo real em favor da facilidade de acesso aos dados relevantes para o seu estudo atual.
 - As aulas de história são essencialmente abstrações de histórias locais e biografias individuais ensinadas como história nacional ou mundial – projeções abstratas de eventos do mundo real.
- A *decomposição de problemas* é um método para desmontar problemas e dividi-los em problemas menores e maiores constituintes compreensíveis. Este método também é conhecido como “Dividir para Conquistar”.
- *Pensamento Algorítmico* é o processo de construção de um esquema de etapas ordenadas que podem ser seguidas para fornecer soluções para todos os problemas constituintes necessários para resolver o problema original.
- *Automação* é a configuração de algoritmos formados sobre computadores e recursos tecnológicos a serem eficientemente aplicável a outros problemas.

- **Generalização** é o processo de adaptação de soluções formuladas ou algoritmos em diferentes estados de problemas, mesmo que as variáveis envolvidas sejam diferentes.

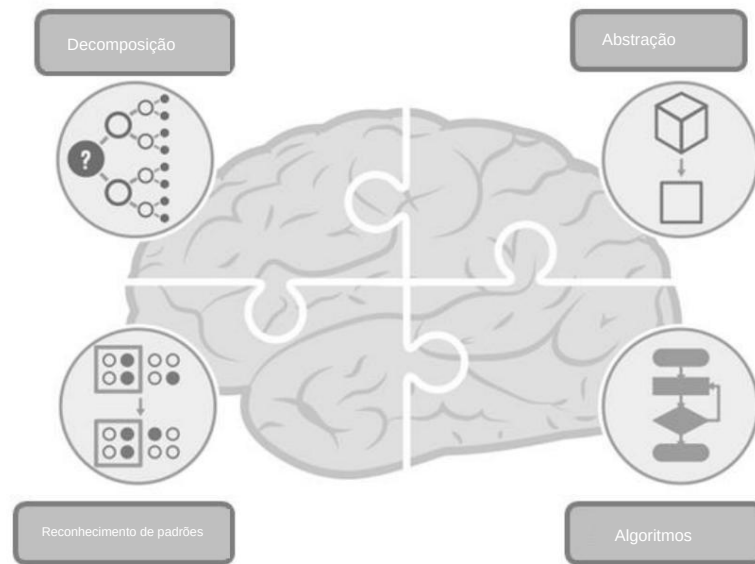


Figura 2. 4 estratégias básicas para pensamento computacional (McNicholl, 2018: p.37).

Existem também várias técnicas usadas para exemplificar e avaliar o pensamento computacional. Estes constituem o equivalente a um método científico para a ciência da computação. Eles são empregados para colocar o pensamento computacional em prática na sala de aula, em casa e no trabalho (Humphreys, 2015):

- **Reflexão**
 - Reflexão é a habilidade de fazer julgamentos (avaliações) justos e honestos em situações complexas que não são isentas de valores. Dentro da ciência da computação, esta avaliação é baseada em critérios usados para especificar o produto, heurísticas (ou regras práticas) e necessidades do usuário para orientar os julgamentos. A percepção de uma criança, ao brincar com pedras, de que $3 + 4$ é igual a $4 + 3$ é um exemplo de reflexão (ou melhor, de abstração reflexiva). A informação criada neste exemplo não deriva das pedras em si, mas das ações realizadas sobre elas.
- **Codificação**
 - Um elemento essencial do desenvolvimento de qualquer sistema de computador é traduzir o projeto em forma de código e avaliá-lo para garantir que funcione corretamente sob todas as condições previstas. A depuração é a aplicação sistemática de análise e avaliação usando habilidades como teste, rastreamento e pensamento lógico para prever e verificar resultados.
 - **Projetando**
 - Projetar envolve trabalhar a estrutura, a aparência e a funcionalidade dos artefatos. Envolve a criação de representações do design, incluindo representações legíveis por humanos, como fluxogramas, storyboards, pseudocódigo, diagramas de sistemas, etc. Envolve outras atividades de decomposição, abstração e design de algoritmos.
- **Analisando**
 - Analisar envolve dividir em partes componentes (decomposição), reduzir a complexidade desnecessária (abstração), identificar os processos (algoritmos) e buscar pontos em comum ou padrões (generalização). Envolve usar o pensamento lógico para entender melhor as coisas e avaliá-las como adequadas ao propósito.
- **Candidatura**
 - Aplicar é a adoção de soluções pré-existentes para atender às necessidades de outro contexto.
É na generalização – a identificação de padrões, semelhanças e conexões – e na exploração dessas características da estrutura ou função dos artefatos. Um exemplo inclui o desenvolvimento de um subprograma ou algoritmo em um contexto que pode ser reutilizado em um contexto diferente.

5. Crítica e Pesquisa Contemporânea em Pensamento Computacional

Wing (2006), no artigo "Pensamento Computacional", forneceu uma definição de pensamento computacional, e sustentou que o pensamento computacional é uma habilidade fundamental para o futuro que se tornará uma necessidade para todos os indivíduos e deve ser empregado nos currículos de estudantes de todos os níveis. No entanto, o próprio artigo de Wing (2006) totalizou apenas 4 páginas, não foi baseado em pesquisas independentes e careceu de análise aprofundada de muitos tópicos abordados no artigo. Embora o artigo tenha sido usado como base para pesquisas feitas por muitos acadêmicos, ele também foi alvo de muitas críticas. Hemmendinger (2010) afirmou especialmente que os componentes do pensamento computacional apresentados em Wing (2006) não são exclusivos do pensamento computacional. De acordo com Hemmendinger (2010):

- A reformulação de problemas difíceis é típica de todos os domínios da resolução de problemas,
- Os filósofos têm pensado em pensar — recursivamente — há muito tempo,
- A matemática certamente usa abstração, assim como todas as disciplinas que constroem modelos,
- A separação de preocupações e o uso de heurísticas também caracterizam a resolução de problemas em geral.,

Além disso, Hemmendinger (2010) avança que ensinar indivíduos envolvidos em outras disciplinas como pensar como um cientista da computação não é razoável. Em vez de empregar uma única disciplina para ditar os processos de pensamento para todas as disciplinas, os físicos deveriam pensar como físicos e os economistas deveriam pensar como economistas, *enquanto* fazendo uso de pensamento computacional e tecnologias de processamento computacional para resolver questões em seu campo de forma eficiente e determinar novas questões que resultariam em métodos novos e eficientes, uma vez resolvidos. Outra objeção a Wing vem de Denning (2016). Segundo Denning (2016), o artigo atribui um peso imerecidamente significativo aos algoritmos e ao pensamento algorítmico. Em vez de valorizar os algoritmos acima da sua contribuição, Denning (2016) sugere que um modelo de pensamento computacional controlado por algoritmos não deve ser ignorado como alternativa. Além disso, eles avançam a noção de que o pensamento computacional não é uma habilidade fundamental e não pode ser considerado igual a habilidades fundamentais como leitura e escrita. Em suma, a ideia de que cada indivíduo pode utilizar o pensamento computacional e campanhas com afirmações como "Programação para Todos", "Uma Nação de Codificadores" e "Um Codificador em Cada Casa" não é realista. A questão de saber se cada profissão e cada indivíduo realmente precisa empregar o pensamento computacional e as consequentes habilidades de codificação como parte do pensamento computacional é uma discussão não resolvida na área. Um dos comentários mais marcantes sobre este enigma é fornecido por Barr & Stephenson (2011: p.113):

O objetivo final não deveria ser ensinar todos a pensar como um cientista da computação, mas sim ensiná-los a aplicar esses elementos comuns para resolver problemas e descobrir novas questões que podem ser exploradas dentro e através de todas as disciplinas (Barr e Stephenson, 2011: p. .113).

Aprender pensamento computacional e ciência da computação não são a mesma coisa. No entanto, coloquialmente, essas duas expressões são usadas de forma intercambiável. Esta suposta equivalência é errada, uma vez que esta última se destina essencialmente a educar os alunos no estudo e utilização dos princípios do cálculo matemático. Uma razão pela qual esta crença está em ampla circulação poderia ser a afirmação original de Wing (2006) de que "o pensamento computacional é pensar como um cientista da computação". Denning (2009) e Hemmendinger (2010) opõem-se a esta afirmação principalmente devido à sua tese de que tal definição de pensamento computacional poderia dar a impressão de que o pensamento computacional só é relevante para o campo da ciência da computação e é amplamente inaplicável a situações cotidianas em situações normais. ser aprendizes de pensamento computacional.

A educação em programação é um subcampo da ciência da computação e, embora seja conduzida principalmente para educar os alunos nas melhores práticas de programação de computadores, um de seus objetivos é conduzir à criação de programas de computador de alta qualidade. O pensamento computacional, embora tenha uma sobreposição considerável com a ciência da computação em certos elementos, concentra-se principalmente no desenvolvimento e na disseminação de abordagens para a resolução de problemas, ao contrário da ciência da computação.

Embora os termos "codificação" e "programação" sejam usados indistintamente, "codificação" tem sido empregada como uma alternativa mais excitante e menos assustadora, especialmente para atrair e motivar iniciantes em scripts. Plataformas como Code Studio, Hour of Code, Code Monkey e Scratch do MIT e App Inventor 2 tendem a usar codificação em vez de programação. Linguagens OOP e baseadas em texto mais avançadas (Python, Java etc.) preferem o uso de programação. Uma crença amplamente difundida é que o pensamento computacional e, como resultado, a educação em codificação e programação, tem um efeito positivo nas habilidades de resolução de problemas dos alunos. Múltiplas manifestações diferentes desta crença podem ser observadas na investigação contemporânea, e podem ser ligadas a um raciocínio científico mais sólido através da análise dos resultados da investigação contemporânea:

- O estudo de meta-análise de Palumbo (1990) concluiu que não foi possível encontrar fortes evidências da existência de uma correlação significativa entre o ensino de programação e as capacidades de resolução de problemas. Palumbo (1990) chegou a esta conclusão avaliando diferentes estudos realizados com estudantes do ensino médio por uma

variedade de pesquisadores. Estes incluíram estudos baseados em CAI (Instrução Assistida por Computador), LOGO e Línguas BASIC sendo ensinadas a diferentes grupos de alunos em várias horas de aula e duração total do curso em configurações de semanas – nenhuma das quais descobriu uma correlação cientificamente significativa. Conforme afirmado anteriormente neste artigo, uma das razões para a quase extinção destas linguagens de programação pode ser a sua incapacidade de fornecer a contribuição esperada para a capacidade de resolução de problemas dos alunos.

- Kalelioglu & Gülbahar (2014) realizaram um estudo de 5 semanas com alunos do 5º ano do ensino secundário (22 raparigas e 27 rapazes) no ano lectivo de 2013-2014. Os alunos realizaram diversas atividades na linguagem de programação Scratch como parte do estudo. Seus resultados indicaram que quando os dados quantitativos são analisados, não houve divergência estatisticamente significativa entre os quocientes de capacidade de resolução de problemas pré-estudo e pós-estudo. A análise dos dados qualitativos, por outro lado, mostrou um maior entusiasmo dos alunos pela programação.
- Kukul & Gökçearslan (2014) trabalharam com 304 alunos do 5º e 6º ano que não tinham tido aulas de programação anteriormente. Da mesma forma que Kalelioglu & Gülbahar (2014), também utilizaram Scratch. Suas conclusões indicaram que não foi observada nenhuma mudança estatisticamente significativa nas habilidades de resolução de problemas dos alunos.
- Morelli et al. (2011) analisaram os resultados sob indicadores específicos. O aplicativo de programação móvel “App Inventor” foi ensinado a alunos do ensino médio como parte de um programa de verão. Nem os indicadores “*aprendizagem orientada para problemas*” nem “*apoio à aprendizagem*” mencionam um aumento na capacidade de resolução de problemas dos alunos, optando antes por focar no aumento da motivação observado.
- Wong et al. (2015) conduziram um estudo experimental em 264 5^o Alunos da série em Hong Kong entre os anos de 2012 e 2014. O primeiro ano de estudo foi usado para ensinar KODU (um motor de jogo desenvolvido pela Microsoft) aos alunos, enquanto no segundo ano Scratch e Small Basic foram usados no currículo. A média das notas de matemática dos alunos passou de 74,86 em 2012-2013 para 77,59 em 2013-2014. Também foram avaliadas a criatividade, o pensamento crítico e a capacidade de resolução de problemas dos alunos. Com base nos resultados do teste t realizados com base em dados recuperados do portal de avaliação de estudantes da ESDA, a capacidade de resolução de problemas dos estudantes pareceu aumentar de 2,75 para 2,95. No entanto, embora os investigadores tenham indicado que a participação na codificação desenvolveu certas capacidades nos alunos, outras capacidades fundamentais não foram afetadas de forma conclusiva.

Várias afirmações fortes foram feitas sobre a influência positiva da educação em programação/codificação no desenvolvimento cognitivo das crianças. Papert (1980) acreditava que a programação permitia que as crianças moldassem seus próprios ambientes de aprendizagem. A afirmação mais importante de Papert era que aprender LOGO melhorava as habilidades de resolução de problemas, proporcionando experiências concretas que conduziam à conceitualização de imagens em uma escala operacional (como o próprio Papert era um matemático, seus exemplos eram frequentemente baseados em matemática e geometria).

Experiências concretas foram definidas como o aparecimento de formas geométricas na tela.). O pensamento operacional formal foi definido por Piaget como a capacidade de construir relações, fazer inferências e construir hipóteses (Kyncal & Yazgan, 2010: p.724). Um indivíduo capaz de pensamento operacional formal pode fazer abstrações, compreender construções matemáticas que exigem pensamento de alto nível, generalizar aplicando as aquisições destes problemas a outros problemas, é capaz de fazer planos e emprega um método processual de pensamento. Neste ponto, as semelhanças entre o pensamento operacional formal, conforme definido por Piaget, e as habilidades baseadas na TC tornam-se aparentes. É por isso que Papert afirmou que o LOGO poderia ajudar a descartar atitudes negativas em relação à matemática nos alunos, ensinando conceitos matemáticos e fortalecendo o autocontrole e as atitudes orientadas para o sucesso nas crianças.

(Liao & Bright, 1991: p.252).

Os resultados desses estudos mostram opiniões conflitantes na literatura sobre pensamento computacional quando se trata da questão de saber se o ensino de programação por si só tem um efeito significativo nas habilidades de resolução de problemas dos alunos. Mas estudos onde são empregados componentes do pensamento computacional mostram um aumento nas habilidades de resolução de problemas, pensamento abstrato, solução de problemas e aprendizagem cooperativa dos alunos.

- Roman-Gonzales et al. (2017) estudaram 1.251 alunos espanhóis do 5º ao 10º ano. CTt (Teste de Pensamento Computacional) e PMat (Teste de Habilidades Mentais Primárias) foram aplicados aos alunos. A correlação entre habilidades de PC e “memória espacial”, “raciocínio” e “resolução de problemas” foi calculada experimentalmente, sendo k memória espacial ($r=0,44$), raciocínio ($r=0,44$) e resolução de problemas ($r=0,67$). A resolução de problemas parece ser mais influenciada do que outras habilidades.
- Grover, Pea & Cooper (2015) trabalharam com 54 estudantes no norte da Califórnia com idades entre 11 e 14 anos. Um curso de 7 semanas foi projetado para os alunos onde eles usaram a plataforma de codificação Scratch e foram capazes de traduzir seu código em plataformas baseadas em texto com base em suas aquisições na plataforma. Os pesquisadores conseguiram correlacionar as habilidades de CT com as habilidades de resolução de problemas. Quando os resultados são analisados, os alunos demonstram ter avançado especialmente nas habilidades de pensamento algorítmico. Outro ponto interessante é que as experiências anteriores de PC dos alunos e

as habilidades matemáticas (determinadas por um exame introdutório realizado pelas pesquisas) foram fortes indicadores dos resultados da aprendizagem. Pea & Kurland (1984, p.35) enumeraram “habilidade matemática”, “capacidade de memória”, “habilidade de raciocínio analógico”, “habilidade de raciocínio situacional” e “habilidade de pensamento processual” como as habilidades matemáticas necessárias para aquisição de habilidade de programação, enquanto especificando também que os alunos que são especialmente capazes de operar com sucesso a linguagem LOGO também tiveram sucesso nas aulas de inglês e humanidades, e não apenas em matemática.

- Webb (2010) avaliou a contribuição do ensino de programação para as habilidades de solução de problemas dos alunos. Foi planejado um regime de 2 horas por semana durante 5 semanas; com as habilidades de CT conectadas à capacidade de resolução de problemas. Embora 19 rapazes e 21 raparigas estivessem presentes no início, por motivos pessoais e exames, apenas 24 alunos (16 rapazes, 8 raparigas) completaram o regime. Ao final do estudo, os alunos foram solicitados a “Consertar o Programa Frogger” em 40 minutos. Apenas 1 aluno foi reprovado nesta tarefa, e os demais seguiram para a fase de depuração.
- O estudo realizado por Bers et al. (2013) baseou-se em 3 turmas de pré-escola (2 públicas e 1 privada) de 53 alunos no total, e teve uma duração de 20 horas. Durante este estudo, os alunos foram expostos a 6 assuntos principais, incluindo processos de projeto de engenharia, robótica, programação baseada em instruções, loops, sensores e mecanismos de controle. Robôs e software TangibleK foram empregados no estudo. Os conteúdos destas disciplinas foram adaptados à idade dos alunos. Músicas, jogos e movimentos rítmicos e repetitivos foram inseridos nos aplicativos. Por exemplo, “Simon Says” foi usado na lição 3: programação algorítmica e CHERP (Creative Hybrid Environment for Robotic Programming), um software de arrastar e soltar foi ensinado. Foram estudadas a solução de problemas dos alunos, a compreensão da relação entre instruções e movimento e o uso da ordem das instruções e das instruções de controle de fluxo. Os resultados indicaram que as capacidades dos alunos para cooperar, criar ideias, partilhar através de negociação, bem como melhoraram as capacidades motoras. Além disso, foi descrito que os alunos se tornaram mais ativos na sua criatividade e capacidade de resolução de problemas, tanto no mundo matemático como no mundo real.
- O estudo realizado por Durak e Sarýtepeci (2018) foi aplicado a 156 estudantes de escolas públicas em Ancara. Duas ferramentas diferentes de coleta de dados foram utilizadas neste estudo. O primeiro é o formulário de informações pessoais e o segundo é o formulário de capacidade de pensamento computacional. Neste estudo, foram examinados os fatores que afetam as habilidades de pensamento computacional dos alunos. Estes factores são o género, o nível de escolaridade, a experiência de utilização de TI, o período de utilização diária da Internet, o desempenho em matemática, as atitudes em relação ao curso de matemática, as atitudes em relação aos cursos de ciências, os resultados nos cursos de ciências, os resultados nos cursos de tecnologias de informação e as atitudes em relação aos cursos de tecnologias de informação. Entre estes factores, foi determinado que os factores mais eficazes nas competências computacionais foram o nível de escolaridade, o aproveitamento em matemática, a atitude em relação ao curso de matemática e a atitude em relação aos cursos de ciências.

Após a análise destes estudos, torna-se evidente que são as aulas de codificação, matemática, ciências naturais, ciências sociais e artes da linguagem, ensinadas de acordo com as competências de pensamento computacional e não a mera programação ou educação de codificação, que afectam um aumento na capacidade de resolução de problemas. , pensamento abstrato, solução de problemas, pensamento processual e habilidades semelhantes nos alunos. É necessário avançar numa aplicação apropriada e interdisciplinar da componente de competências de PC, a fim de formar os alunos não apenas como codificadores, mas como indivíduos com uma forma radical de pensamento e perspectiva. Além disso, pode ser apropriado que o pensamento computacional e o STEAM (Ciência, Tecnologia, Engenharia, Artes e Matemática) sejam considerados em conjunto, uma vez que estes dois campos partilham uma grande quantidade de material temático (Güllbahar, 2017: p.331). Trabalho interdisciplinar por parte dos alunos e a sua capacidade de perceber relações entre áreas de estudo, determinar os problemas que enfrentam, investigar soluções potenciais, decidir sobre a solução correta, recolher dados, analisar dados, resolver problemas, desenvolver os seus modelos e generalizar soluções (ISTE, 2016) ajudará os seus habilidades de resolução de problemas.

6. Conclusão

As tecnologias baseadas na ciência da computação estão a desenvolver-se rapidamente na nossa era, influenciando os processos de resolução de problemas e a vida social dos indivíduos e das sociedades. Do trabalho médico ao uso das mídias sociais, os resultados dos estudos de ciência da computação são integrados à vida cotidiana dos indivíduos em diversas áreas. Os efeitos da ciência da computação na sociedade moderna são também um indicador de seus efeitos no método científico e, portanto, naturalmente, nos cientistas. Os cientistas naturais há muito posicionam a computação como um “terceiro” fundamento do método científico ao lado da teoria e da experimentação, e que o pensamento computacional é essencial para o seu trabalho (Denning, 2009).

Embora as definições e a estrutura do pensamento computacional estabelecidas por Wing (2006) tenham sido criticadas há muito tempo por outros pesquisadores, a importância da ciência da computação tem crescido diariamente, encontrando aplicações em vários campos, desde a cura de doenças até a prevenção de ataques terroristas. No entanto, a afirmação de que a ciência da computação

e como resultado o pensamento computacional é uma disciplina fundamental, a par da leitura, da escrita e da aritmética básica, ainda está em debate.

Noções populistas como “Ciência da Computação e Pensamento Computacional para Todos”, destinadas a trazer o campo para o mainstream, tornarão mais difícil para o campo preservar o seu devido rigor. Como deduzimos dos trabalhos de Denning, Hemmendinger e Barr, entre outros apresentados neste artigo, atribuindo uma importância imerecida a certos campos – sejam eles considerados codificação, ciência da computação ou pensamento computacional –

seria inapropriado. Ainda assim, os investigadores podem beneficiar de considerar o pensamento computacional como um método potencial de transformar a educação, desde que também considerem as críticas aplicadas ao campo com igual consideração. Como Denning (2010, p.28) também afirmou, manter o pensamento computacional (e a codificação) em (imerecidamente) estima excessiva pode nos levar de volta às mesmas armadilhas que estamos tentando evitar.

Como observação final, sustentamos que o objetivo fundamental do pensamento computacional (e de incutir esta capacidade nos alunos) e da educação informática deve ser ajudar os alunos a compreender e – através do uso dos seus impulsos criativos – a mudar o mundo em que vivem, para melhor. (Departamento de Educação, 2014, p.217).

Referências

- Agalianos, A., Noss, R., & Whitty, G. (2001). Logotipo nas escolas regulares: a luta pela alma de um inovação educacional. *Jornal Britânico de Sociologia da Educação*, 22(4), 479-500.
- Angeli, C., Voogt, J., Fluck, A., Webb, M., Cox, M., Malyn-Smith, J., & Zagami, J. (2016). Uma estrutura curricular de pensamento computacional K-6: Implicações para o conhecimento do professor. *Jornal de Tecnologia Educacional e Sociedade*, 19(3), 47.
- Barr, V. e Stephenson, C. (2011). Trazendo o pensamento computacional para o ensino fundamental e médio: o que está envolvido e qual é o papel da comunidade de educação em ciência da computação?. *Acm Inroads*, 2(1), 48-54.
- BCS, Instituto Chartered de TI. 2014. Chamada para apresentação de provas - Grupo de Trabalho de Competências Digitais do Reino Unido. <http://bit.ly/1Ii8mdo> [Recuperado em 17.01.2018].
- Bers, MU, Flannery, L., Kazakoff, ER e Sullivan, A. (2014). Pensamento e ajustes computacionais: Exploração de um currículo de robótica para a primeira infância. *Computadores e Educação*, 72, 145-157.
- Bringuier, JC (1980). Conversas com Jean Piaget. *Sociedade*, 17(3), 56-61.
- Çýnar, M. & Tüzün, H. (2017, fevereiro). Bilgisayýmsal Düyünme Sürecinin Doýasýna ýlýýkin Nitel Bir Analiz (Uma análise qualitativa sobre a natureza do processo de pensamento computacional). Apresentado a 19. Akademik Biliým Konferasý (Conferência sobre Informática Acadêmica), Universidade Aksaray, recuperado em 24.12.2017 em <http://ab.org.tr/ab17/ozet/233.html>.
- Denner, J., Werner, L. e Ortiz, E. (2012). Jogos de computador criados por meninas do ensino médio: eles podem ser usados para medir a compreensão dos conceitos da ciência da computação?. *Computadores e Educação*, 58(1), 240-249.
- Denning, PJ (2009). A profissão de TI Além do pensamento computacional. *Comunicações da ACM*, 52(6), 28-30.
- Departamento de Educação. 2014. O Currículo Nacional na Inglaterra, Documento-Quadro. Referência: DFE-00177-2013. Recuperado em 26.12.2017 em: https://www.gov.uk/government/uploads/system/uploads/attachment_data/file/335116/Master_final_national_curriculum_220714.pdf.
- Durak, HY e Saritepeci, M. (2018). Análise da relação entre habilidades de pensamento computacional e diversas variáveis com o modelo de equações estruturais. *Computadores e Educação*, 116, 191-202.
- Etherington, C. (2017), recuperado em 24.12.2017 em: <https://news.elearninginside.com/how-plato-changed-the-mundo-em-1960/>.
- Furber S (2012) Desligar ou reiniciar? O futuro da computação nas escolas do Reino Unido. Relatório técnico, The Royal Sociedade, Londres.
- Grover, S., Pea, R. e Cooper, S. (2015). Projetando para um aprendizado mais profundo em um curso misto de ciência da computação para alunos do ensino médio. *Educação em Ciência da Computação*, 25(2), 199-237.
- Hemmendinger, D. (2010). Um apelo à modéstia. *Incurões de Acm*, 1(2), 4-7.
- Humphreys, S. (2015). Pensamento Computacional, um guia para professores. Computação na Escola. Charlotte BCS. O Chartered Institute para TI
- ISTE (2011), Definições operacionais de pensamento computacional, recuperado em 24.12.2017 de: <https://c.ymcdn.com/sites/www.csteachers.org/resource/resmgr/CompThinkingFlyer.pdf>.

- ISTE (2016). ISTE Standarts for Students, recuperado em 24.12.2017 em: http://www.iste.org/docs/Standards-Resources/iste-standards_students-2016_one-sheet_final.pdf?sfvrsn=0.23432948779836327.
- Kalelioglu, F. e Gülbahar, Y. (2014). Os efeitos do ensino de programação via Scratch nas habilidades de resolução de problemas: uma discussão a partir da perspectiva dos alunos. *Informática na Educação*, 13(1), 33.
- Kalelioglu, F., Gülbahar, Y., & Kukul, V. (2016). Uma estrutura para pensamento computacional baseada em uma sistemática revisão de pesquisa. *Baltic Journal of Modern Computing*, 4(3), 583.
- Keser, H. (2011). Türkiye'de Bilgisayar Eýitiminde ýlk Adým: Orta Öýretimde Bilgisayar Eýitimi ýhtisas Komisyonu Raporu (Primeiros passos da Turquia na educação em informática: Relatório da Comissão Especializada em Educação em Informática no Ensino Secundário). *Eýitim Teknolojisi Kuram ve Uygulama (Tecnologias Educacionais Teóricas e Práticas)*, 1(2), 83-94.
- Kýncal, RY e Yazgan, AD (2010). Investigar as habilidades de pensamento operacional formal de alunos do 7º e 8º ano do ensino fundamental de acordo com algumas variáveis. *Educação Elementar Online*, 9(2), 723-733.
- Korkmaz, Ö., Çakýr, R., Özden, MY, Oluk, A., & Sarýoýlu, S. (2015). Bireylerin Bilgisayarca Dýýünme Becerilerinin Farklý Deýiýkenler Açýsýndan ýncelenmesi (Uma investigação multivariável das habilidades de pensamento computacional dos indivíduos). *Ondokuz Mayýs Üniversitesi Eýitim Fakóltesi Dergisi (19º Jornal da Faculdade de Educação da Universidade de Maio)*, 34(2), 68-87.
- Korkmaz, Ö., Çakýr, R., Özden, MY, Oluk, A., & Sarýoýlu, S. (2015). Bireylerin Bilgisayarca Dýýünme Becerilerinin Farklý Deýiýkenler Açýsýndan ýncelenmesi (Uma investigação multivariável das habilidades de pensamento computacional dos indivíduos). *Ondokuz Mayýs Üniversitesi Eýitim Fakóltesi Dergisi (19º Jornal da Faculdade de Educação da Universidade de Maio)*, 34(2), 68-87.
- Kukul, V., & Gökçearslan, ý. (2014). Scratch ile programlama eýitimi alan öýrencilerin problem çözmeye becerilerinin incelenmesi. (Investigação das habilidades de resolução de problemas de alunos com educação em programação baseada em Scratch.)
- Lee, I., Martin, F., Denner, J., Coulter, B., Allan, W., Erickson, J., ... & Werner, L. (2011). Pensamento computacional para jovens na prática. *Incursões de Acm*, 2(1), 32-37.
- Liao, YKC e Bright, GW (1991). Efeitos da programação de computadores nos resultados cognitivos: uma meta-análise. *Jornal de Pesquisa em Computação Educacional*, 7(3), 251-268.
- Fundação Logo (2015). Logo and Learning, recuperado em 24.12.2017 em: http://el.media.mit.edu/logo-foundation/what_is_logo/logo_and_learning.html.
- McNicholl, R.(2018). Pensamento computacional usando code.org. Olá mundo, 4, 37.
- Morelli, R., De Lanerolle, T., Lake, P., Limardo, N., Tamotsu, E., & Uche, C. (2011, março). O inventor do aplicativo Android pode trazer o pensamento computacional para o ensino fundamental e médio. Em *Proc. 42º Simpósio Técnico ACM sobre Ensino de Ciência da Computação (SIGCSE'11)* (s. 1-6).
- Conselho Nacional de Pesquisa. (2010). Comitê dos Workshops de Pensamento Computacional. No *relatório de um workshop sobre o escopo e a natureza do pensamento computacional*, Natl Academy Pr.
- Palumbo, DB (1990). Pesquisa em linguagem de programação/solução de problemas: uma revisão de questões relevantes. *Revisão de pesquisa educacional*, 60(1), 65-89.
- Papert, S. e Harel, I. (1991). Situando o construcionismo. *Construcionismo*, 36(2), 1-11.
- Ervilha, RD e Kurland, DM (1984). Sobre os efeitos cognitivos da aprendizagem de programação de computadores. *Novas ideias em psicologia*, 2(2), 137-168.
- Román-González, M., Pérez-González, JC e Jiménez-Fernández, C. (2016). Quais habilidades cognitivas estão subjacentes ao pensamento computacional? Validade de critério do Teste de Pensamento Computacional. *Computadores no Comportamento Humano*, 1-14
- Selby, C. e Woollard, J. (2013). Pensamento computacional: a definição em desenvolvimento.
- Tekerek, M. e Altan, T. (2014). O efeito do ambiente scratch no desempenho do aluno no ensino de algoritmos. *Jornal Mundial sobre Tecnologia Educacional*, 6(2), 132-138.
- Voogt, J., Fisser, P., Good, J., Mishra, P., & Yadav, A. (2015). Pensamento computacional na escolaridade obrigatória: Rumo a uma agenda de investigação e prática. *Educação e Tecnologias de Informação*, 20(4), 715-728.
- Asa, J. (2014). O pensamento computacional beneficia a sociedade. *Blog do 40º aniversário de questões sociais em computação*, 2014.
- Asa, JM (2006). Pensamento computacional. *Comunicações da ACM*, 49(3), 33-35.

- Asa, JM (2008). Pensamento computacional e pensamento sobre computação. *Transações filosóficas da sociedade real de Londres A: ciências matemáticas, físicas e de engenharia*, 366(1881), 3717-3725.
- Wing, JM (2011), Caderno de Pesquisa: Pensamento computacional - o quê e por quê? A Revista Link, 20-23.
<https://www.cs.cmu.edu/link/research-notebook-computational-thinking-what-and-why>
- Yadav, A., Mayfield, C., Zhou, N., Hambruch, S., & Korb, JT (2014). Pensamento computacional na formação de professores do ensino fundamental e médio. *Transações ACM em Educação em Computação (TOCE)*, 14(1), 5.
- Yecan, E., Özçınar, H., & Tanyeri, T. (2017). Biliyim Teknolojileri Öğretmenlerinin Görsel Programlama Öğretimi Deneyimleri (Uma coleção de experiências de programação visual por educadores de tecnologias da informação). *İlköğretim Online (Ensino Elementar Online)*, 16(1).